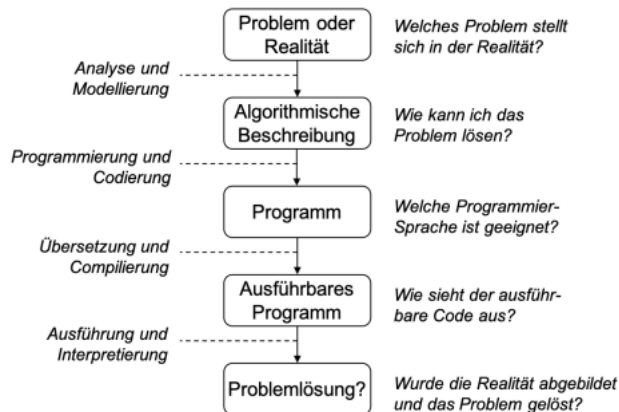


A. Einführung

1. Ziel der Programmierung: Umsetzung eines gegebenen oder selbstentwickelten Algorithmus in ein lauffähiges Computerprogramm.



2. Algorithmus

+ **Definition:** ist eine präzise endliche Beschreibung eines allgemeinen Verfahrens unter Verwendung ausführbarer elementarer (Verarbeitungs-)Schritte.

+ **Eigenschaften:**

→ **Terminierung:** bricht nach endlich vielen Schritten ab.

→ **Determinismus:**

- Legt die Wahlfreiheit fest
- **Deterministischer Ablauf:** legt eindeutige Vorgabe der Schrittfolge der auszuführenden Schritte fest.
- **Determiniertes Ergebnis:** bei vorgegebener Eingabe wird ein eindeutiges Ergebnis geliefert – auch bei mehrfacher Durchführung mit denselben Eingabeparametern.

+ **Bestandteile:**

- Elementare Operationen
- Sequenzielle Ausführung
- Parallele Ausführung
- Bedingte Ausführung
- Schleife
- Unterprogramm (bsp. Reche Aufgabe 1 anhand der Lösung auf Seite 100)
- Variablen und Konstanten

3. Ausdruck:

+ **Kombination von Operanden und Operatoren als „Vorschrift“ zur Berechnung eines Werts.**

+ liefert immer einen Wert ab.

+ Bsp.: $1/x$

4. Anweisung:

+ **Kombination von Ausdrücken und Methoden als „Vorschrift“ zur Ausführung einer Aktion.**

+ Bsp.:

$Y=1/x$ Wertzuweisung

Print(x) Ausgabeanweisung

5. Sequenz:

- + bildet eine zeitliche Abfolge von Anweisungen
- + einzelne Schritte werden durchnummeriert oder es wird zum Abschluss der Sequenz ein Semikolon gesetzt.

6. Bedingte Anweisung:

- + es werden Bedingungen auf ihre Richtigkeit geprüft.
- + für wahre und falsche Aussagen in der Bedingung können unterschiedliche Anweisungen ausgeführt werden

7. Schleifen:

- + bestimmte Anweisungen werden wiederholt, bis eine definierte Endbedingung erfüllt wird.
- + Unterscheidung in drei Schleifenarten.

8. Unterprogramme:

- + beinhaltet einen Teilalgorithmus
- + dieser Teilalgorithmus kann in mehreren Algorithmen wieder verwendet werden.

9. Variablen:

- + „Platzhalter“ für einen konkreten Wert
- + sind von einem bestimmten Datentyp können ihren Wert ändern.

10. Konstanten:

- + haben einen festen Wert
- + sind von einem bestimmten Datentyp
- + könne ihren Wert nicht ändern

11.

Darstellungsformen von Algorithmen

Pseudocode

- nahe an den Konstrukten verbreiteter Programmiersprachen
- Verwendung spezieller englischer Begriffe aus dem Alltag
- Begriffe haben eine festgelegte Bedeutung

Programmablaufpläne

- genormt nach DIN 66001
- Ursprung in der linearen Programmierung
- nur für kleinere Programme geeignet (Übersichtlichkeit)

Nassi-Schneiderman-Diagramme (Struktogramme)

- Entstehung 1973
- Darstellung genormt nach DIN 66261 im Jahr 1985
- überwiegender Einsatz in der prozeduralen Programmierung

Pseudocode

Sequenz

- Alternative 1: Schritte werden durchnummeriert: 1, 2, 3, ...
- Alternative 2: Abschluss der Sequenz durch Semikolon
- Vorteil Alternative 1: Verfeinerung einzelner Schritte: 2.1, 2.2, ...

Bedingte Anweisung

- es werden Bedingungen auf Ihre Richtigkeit geprüft
- Alternative 1: falls Bedingung dann Schritt
- Alternative 2: falls Bedingung dann Schritt A sonst Schritt B

Schleifen

- kopfgesteuert: solange Bedingung wahr führe aus Schritte
- fußgesteuert: wiederhole Schritte bis Bedingung wahr
- Zählschleife: wiederhole für Zahlenbereich Arbeitsschritte

Beispiel: Pseudocode

Sequenz

1. Koche Wasser
2. Gib Kaffeepulver in Tasse
3. Fülle Wasser in Tasse

Bedingte Anweisung

falls Ampel rot oder gelb
dann stoppe
sonst fahre weiter

falls Ampel ausgefallen
dann fahre vorsichtig weiter
sonst falls Ampel grün
dann fahre weiter
sonst stopp

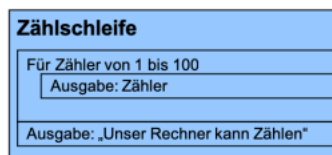
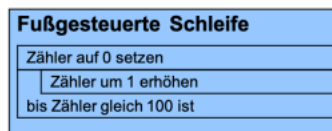
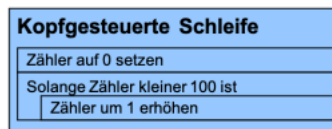
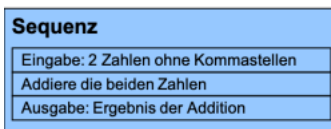
Schleifen

solange Liste nicht erschöpft
führe aus
 Gib nächste Zahl aus der Liste aus

wiederhole
 Gib nächste Zahl aus der Liste aus
bis Liste erschöpft

wiederhole für 5 bis 10
 Gib nächste Zahl aus der Liste aus

Struktogramme



B. Grundlagen von Java

1. Klassifizierung von Software:

- Programme zur Steuerung der Verarbeitungsprozesse, der Übertragungsprozesse und der Speicherungsprozesse in Computern
- unterschiedliche Softwarearten
 - Systemsoftware
 - grundlegende Dienste für andere Programme
 - Steuerung des Computersystems (Hardware)
 - Ablaufsteuerung anderer Programme
 - Entwicklungssoftware
 - Erstellung und Modifikation von Programmen
 - Übersetzungsprogramme für Programmiersprachen
 - Anwendungssoftware
 - Programme zur Verarbeitung der Daten
 - Unterhaltungssoftware
 - Spiele

2. Eigenschaften von Java:

- sicher
 - keine direkten Speicherzugriffe mit * Pointerarithmetik
 - strenge Typüberprüfung
 - keine Programmierung mit Sprachverletzung
- robust
 - keine Rechnerabstürze durch Programmierfehler
 - Überprüfung der Speicherzugriffe
 - Ausnahmeroutinen zur Fehlerbehandlung
- multithreaded
 - parallele Ausführung von Programmteilen
- internetfähig
 - Java-Applets sind über das Internet verteilbar und können lokal auf dem Rechner ausgeführt werden
- vollständig objektorientiert
 - ohne prozedurale Altlasten
- unkompliziert - einfach und leicht erlernbar
 - Syntax ähnlich zu C, C++
 - Beschränkung auf das Notwendigste
 - keine Pointer
 - keine Header-Dateien
 - keine Präprozessor-Anweisungen
 - keine Mehrfachvererbung
- plattformunabhängig (architekturneutral)
 - übersetzte Java-Programme (Bytecode) sind auf jeder javafähigen Plattform ausführbar
 - fest definierter Wertebereich für Zahlen

3. Funktionsweise eines Compiler

- übersetzt ein Computerprogramm aus einer Quellsprache in ein semantisch äquivalentes Programm einer Zielsprache
- Aufbau eines Compilers in 2 Phasen
 - Analysephase
 - lexikalische Analyse zerteilt Code in zusammengehörende Token
 - syntaktische Analyse überprüft auf formale Richtigkeit
 - semantische Analyse überprüft die logischen Rahmenbedingungen
 - Synthesephase
 - Zwischencodeerzeugung liefert die Basis für die Optimierung
 - Programmoptimierung auf Basis des maschinennahen Zwischencodes
 - Codegenerierung erzeugt den Programmcode der Zielsprache

4. Arten von Compilern

- Native Compiler erzeugt Code für die Plattform, auf der er läuft
- Cross-Compiler erzeugt Code für andere Plattformen
- One-pass-Compiler erzeugt den Code in einem Durchlauf
- Multi-pass-Compiler erzeugt den Code in mehreren Durchläufen

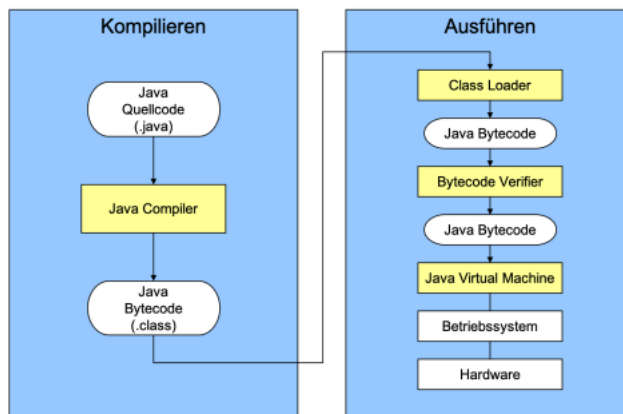
5. Linker:

- stellt einzelne Programmmodule zu einem ausführbaren Programm zusammen
- fügt benötigten Code aus Funktionsbibliotheken zum Code des Hauptprogramms hinzu
- statisches Linken
 - beim kompilieren werden benötigte Codings aus Funktionsbibliotheken dem Coding des Hauptprogramms hinzugefügt
- dynamisches Linken
 - zur Laufzeit werden die Codings aus Funktionsbibliotheken o.ä. zum Hauptprogramm hinzugelinkt
 - DLL-Konzept von Windows
 - auch bei Java findet dynamisches Linken statt

6. Funktionsweise eines Interpreters

- übersetzt den Quellcode nicht in eine direkt ausführbare Datei
- liest den Quellcode ein, analysiert diesen und führt ihn dann aus
- die Analyse erfolgt also zur Laufzeit des Programms
- auf jeder Rechnerarchitektur lauffähig
- deutlich langsamer als kompilierte Programme
- Möglichkeiten zur Steigerung der Geschwindigkeit
 - Just-In-Time-Compiler
 - zur Laufzeit wird der Quellcode in einen Maschinencode übersetzt
 - Maschinencode wird direkt vom Prozessor ausgeführt
 - mehrfach durchlaufene Programmteile müssen nur ein Mal übersetzt werden
 - nur auf einer bestimmten Rechnerarchitektur lauffähig
 - Bytecode-Interpreter
 - Quellcode wird zur Laufzeit in sog. Bytecode übersetzt
 - Bytecode wird von einem Interpreter (Virtual Machine) ausgeführt
 - Bytecode kann auf verschiedenen Plattformen ausgeführt werden

7. Bytecode und virtual Machine



8. Das Paketkonzept in Java

- Möglichkeit zur Strukturierung bzw. sinnvollen Sortierung von Klassen
- Pakete sind somit Sammlungen von Klassen
- die Klassen verfolgen einen gemeinsamen Zweck
- jede Klasse ist genau einem Paket zugeordnet
- Paketnamen können aus mehreren Teilen bestehen und hierarchisch aufgebaut sein (vergleichbar mit der Ordnerstruktur im Windows-Explorer)
- eine Klasse kann eindeutig über das Paket und ihren Namen identifiziert werden

C. Datentypen

1. Arten von Datentypen

- bool'sche Typen sind Wahrheitswerte: True und False
- numerische Typen
 - Byte, Short, Integer und Long: Menge der ganzen Zahlen
 - Double und Float: Menge der reellen Zahlen
- alphanumerisch
 - Char: Menge der vereinbarten Schriftzeichen, z.B. ASCII-Zeichensatz
- komplexe Datentypen
 - String: Verkettung von Character-Werten ⇒ Zeichenketten
 - Array: Definition eines n-dimensionalen Feldes
 - Referenzdatentypen: Individuell definierte Datentypen

2. Wertebereiche und Speicherverbrauch

Typname	Länge in Byte	Wertebereich
boolean	1	<i>true, false</i>
char	2	Alle Unicode-Zeichen
byte	1	-2^7 bis 2^7-1 (-128 bis 127)
short	2	-2^{15} bis $2^{15}-1$ (-32.768 bis 32.767)
int	4	-2^{31} bis $2^{31}-1$ (-2.147.483.648 bis 2.147.483.647)
long	8	-2^{63} bis $2^{63}-1$ (-9.223.372.036.854.775.808 bis 9.223.372.036.854.775.807)
float	4	$\pm 3.40282347 \cdot 10^{38}$
double	8	$\pm 1.79769313486231570 \cdot 10^{308}$

3. Literale:

+ bezeichnet fest definierte Zeichenfolgen zur Darstellung von Werten für Basis-Datentypen.

4. Numerische Literale

Ganzzahlen

Literale für die Familie der Integer-Werte (byte, short, int, long) können in Dezimal-, Oktal- oder Hexadezimalform geschrieben werden

- Dezimalliterale bestehen aus den Ziffern 0 bis 9
- Oktalliterale bestehen aus dem Präfix 0 und den Ziffern 0 bis 7
- Hexadezimaliterale bestehen aus dem Präfix 0x, den Ziffern 0 bis 9 und den Buchstaben A bis F bzw. a bis f
- durch das Suffix l bzw. L wird ein Literal vom Typ long erzeugt

Fließkommazahlen

Literale für Fließkommazahlen float und double

- bestehen aus einem Vorkommateil, dem Dezimalpunkt, einem Nachkommateil, einem Exponenten und einem Suffix
- Unterscheidung zwischen float und double durch das Suffix f bzw. d
- Exponent wird durch ein e bzw. E eingeleitet
- Vorkomma- oder Nachkommateil darf ausgelassen werden, der Exponent und das Suffix sind optional
- ohne Suffix sind Fließkommazahlen immer vom Typ double

5. Alphanumerische Literale

- besondere Literale für den Datentyp boolean

true und false

- Char-Literale werden grundsätzlich in einfache Hochkommata gesetzt
- Char-Variablen sind prinzipiell 2 Byte lang, da in char Unicode-Werte gespeichert werden können

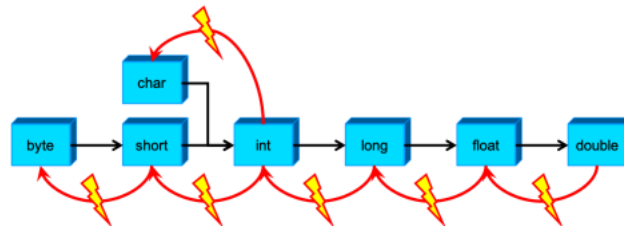
```
'a';  
'\u2764';
```

- besondere Zeichenliterale stellen die String-Literale dar
- String-Literale stehen in doppelten Hochkommata

```
"Hier ist eine Zeichenkette"
```

6. Konvertierungsregeln

- generelle Unterscheidung in erweiternde und einschränkende Konvertierung
- es existiert keine Konvertierungsvorschrift für den Datentyp boolean und für die Konvertierung zwischen einfachen und Referenzdatentypen



7. Arrays

- Arrays sind (mehrdimensionale) Feldvariablen, die aus mehreren Elementen bestehen
- alle Elemente eines Arrays gehören dem gleichen Datentyp an
- in Java sind Arrays semidynamisch, d.h. ihre Größe kann zur Laufzeit festgelegt, aber danach nicht mehr verändert werden
- Arrays in Java sind Objekte und bieten verschiedene Methoden
- die Elemente eines Arrays sind bei n Elementen von 0 bis n-1 durchnummeriert
- Zugriffe auf einzelne Elemente von Arrays erfolgen über den numerischen Index
- das Attribut length ist vom Typ Integer und gibt die Anzahl der Elemente eines Arrays zurück

8. Deklaration und Initialisierung von Arrays

- die Deklaration eines Arrays entspricht syntaktisch der einer einfachen Variablen
- Unterschied: an den Typnamen oder an den Variablennamen werden eckige Klammern angefügt
- die Initialisierung erfolgt mithilfe des new-Operators oder durch Zuweisung eines Array-Literals
- Beispiele

```
int [] zahl = {1, 2, 3, 4, 5};
zahl[1] = 7;
```

```
int [][] matrix = new int[3][3];
matrix[1][2] = 15;
```

```
int [][] matrix = new int[3][];
matrix[0] = new int[4];
```

```
int [][][] wuerfel = new int[3][3][3];
wuerfel[2][0][0] = 73;
```

D. Ausdrücke und Anweisungen

1. Ausdrücke

- kleinste ausführbare Einheit in einem Programm
- Ausdrücke haben folgende Aufgaben
 - Variablen Werte zuzuweisen
 - numerische Berechnungen durchzuführen
 - logische Bedingungen zu formulieren
- Ausdrücke bestehen aus mindestens einem Operator und einem oder mehreren Operanden, auf die der Operator angewendet wird
- Unterscheidung der Operatoren nach dem Typ der Operanden
 - arithmetische Operatoren
 - relationale Operatoren
 - logische Operatoren
 - (bitweise Operatoren)
 - Zuweisungsoperatoren
 - sonstige Operatoren

2. Operatoren

Arithmetische Operatoren

Operator	Bezeichnung	Bedeutung
+	Positives Vorzeichen	+n ist gleichbedeutend mit n
-	Negatives Vorzeichen	-n kehrt das Vorzeichen von n um
+	Summe	a + b ergibt die Summe von a und b
-	Differenz	a - b ergibt die Differenz
*	Produkt	a * b ergibt das Produkt
/	Quotient	a / b ergibt den Quotienten
%	Restwert	a % b ergibt den Rest der ganzzahligen Division von a durch b
++	Präinkrement	++a erhöht a um 1 und ergibt a
++	Postinkrement	a++ ergibt a und erhöht a um 1
--	Prädecrement	--a verringert a um 1 und ergibt a
--	Postdecrement	a-- ergibt a und verringert a um 1

Relationale Operatoren

Operator	Bezeichnung	Bedeutung
==	Gleich	a == b ergibt true, wenn a gleich b ist. Sind a und b Referenztypen, so ist der Rückgabewert true, wenn beide Werte auf dasselbe Objekt zeigen.
!=	Ungleich	a != b ergibt true, wenn a ungleich b ist. Sind a und b Referenztypen, so ist der Rückgabewert true, wenn beide Werte auf verschiedene Objekt zeigen.
<	Kleiner	a < b ergibt true, wenn a kleiner b ist.
<=	Kleiner gleich	a <= b ergibt true, wenn a kleiner oder gleich b ist.
>	Größer	a > b ergibt true, wenn a größer b ist.
>=	Größer gleich	a >= b ergibt true, wenn a größer oder gleich b ist.

Logische Operatoren

Operator	Bezeichnung	Bedeutung
!	Logisches NICHT	!a ergibt false, wenn a wahr ist, und true, wenn a falsch ist.
&&	UND mit Short-Circuit-Evaluation	a && b ergibt true, wenn sowohl a als auch b wahr sind. Ist a bereits falsch, so wird false zurückgegeben und b nicht mehr ausgewertet.
	ODER mit Short-Circuit-Evaluation	a b ergibt true, wenn mindestens einer der beiden Ausdrücke a oder b wahr ist. Ist a bereits wahr, so wird true zurückgegeben und b nicht mehr ausgewertet.
&	UND ohne Short-Circuit-Evaluation	a & b ergibt true, wenn sowohl a als auch b wahr sind. Beide Teilausdrücke werden ausgewertet.
	ODER ohne Short-Circuit-Evaluation	a b ergibt true, wenn mindestens einer der beiden Ausdrücke a oder b wahr ist. Beide Teilausdrücke werden ausgewertet.
^	Exklusiv-ODER	a ^ b ergibt true, wenn beide Ausdrücke einen unterschiedlichen Wahrheitswert haben.

Zuweisungsoperatoren

Operator	Bezeichnung	Bedeutung
=	Einfache Zuweisung	a = b weist a den Wert von b zu und liefert b als Rückgabewert.
+=	Additionszuweisung	a += b weist a den Wert von a + b zu und liefert a + b als Rückgabewert.
-=	Subtraktions-zuweisung	a -= b weist a den Wert von a - b zu und liefert a - b als Rückgabewert.
*=	Multiplikations-zuweisung	a *= b weist a den Wert von a * b zu und liefert a * b als Rückgabewert.
/=	Divisionszuweisung	a /= b weist a den Wert von a / b zu und liefert a / b als Rückgabewert.
%=	Modulozuweisung	a %= b weist a den Wert von a % b zu und liefert a % b als Rückgabewert.

Sonstige Operatoren

- der Fragezeichenoperator ?:
 - ist der einzige dreistellige Operator in Java
 - er besteht aus einem logischen Ausdruck und zwei weiteren Ausdrücken, die entweder numerisch, boolean oder von einem Referenztyp sind.
- ```
a ? b : c // ==> results in b if a is true or c if a is false
```
- Type-Cast-Operator
    - dient der expliziten Typumwandlung bei einer einschränkenden Konvertierungen
    - (type) a wandelt den Ausdruck a in einen Ausdruck vom Typ type um
  - String-Verkettung
    - mit dem +-Operator können nicht nur mit numerische Operanden verwendet werden, sondern auch zur Verkettung von Strings
    - a + b ergibt einen verketteten String, wenn einer der Operanden a oder b ein Objekt vom Typ String ist

### 3. Anweisungen

Anweisungen sind elementare Arbeitsschritte

#### die leere Anweisung

```
;
```

- die einfachste Anweisung in Java
- kann dort verwendet werden, wo syntaktisch eine Anweisung erforderlich, aber von der Programmlogik nicht sinnvoll ist

#### der Anweisungsblock

```
{
 Anweisung 1;
 Anweisung 2;
 ...
 Anweisung n;
}
```

- ist eine Zusammenfassung von Anweisungen
- wird als einzelne Anweisung behandelt und wird dort verwendet, wo syntaktisch nur eine Anweisung erlaubt ist

### 4. Beeinflussung des Kontrollflusses

*Der Kontrollfluss (oder Programmablauf) bezeichnet in der Informatik die zeitliche Abfolge der einzelnen Befehle eines Computerprogramms. Der Kontrollfluss ist gewöhnlich durch die Reihenfolge der Befehle innerhalb des Programms festgelegt. Durch Kontrollstrukturen kann von der sequenziellen Abarbeitung abgewichen werden. Hierzu zählen beispielsweise Verzweigungen und Schleifen.*

## 5. Verzweigungen

### Verzweigungen - if-Anweisung

#### die einfache if-Anweisung

```
if (ausdruck)
 anweisung;
```

- *ausdruck* kann aus einem relationalen Operator oder aus mehreren relationalen Operatoren bestehen, die mit logischen Operatoren verknüpft sind
- *anweisung* wird genau dann ausgeführt, wenn *ausdruck* true ist

#### die if-else-Anweisung

```
if (ausdruck)
 anweisung1;
else
 anweisung2;
```

- *anweisung1* wird ausgeführt, wenn *ausdruck* true ist
- ist *ausdruck* false, wird *anweisung2* ausgeführt

#### die if-else if-Anweisung

```
if (ausdruck1)
 anweisung1;
else if (ausdruck2)
 anweisung2;
else
 anweisung3;
```

- *anweisung1* wird ausgeführt, wenn *ausdruck1* true ist
- ist *ausdruck1* false, wird *ausdruck2* ausgewertet
- ist *ausdruck2* true, wird *anweisung2* ausgeführt
- ist auch *ausdruck2* false, wird *anweisung3* ausgeführt

### Verzweigungen mit der switch-Anweisung

```
switch (ausdruck) {
 case constant1:
 anweisung1;
 case constant2:
 anweisung2;
 // -
 default:
 default_anweisung;}
```

- bietet die Möglichkeit der Mehrfachverzweigung
- Ablauf
  - der *ausdruck* wird zunächst ausgewertet
  - das Ergebnis wird mit den Konstanten der case-Label verglichen und bei Gleichheit die Anweisung und alle nachstehenden Anweisungen ausgeführt (Fall-Through-Logik)
  - dies kann durch Einsatz einer break-Anweisung verhindert werden
  - jede Konstante darf nur einmal auftauchen
  - gibt es keine passende Konstante, wird das optionale default-Label am Ende der switch-Anweisung angesprungen

## 6. Schleifen

- kopfgesteuerte Schleifen
  - die Abbruchbedingung steht im Schleifenkopf
  - wird auch als abweisende Schleife bezeichnet
  - wird 0, 1 oder n Mal durchlaufen
- fußgesteuerte Schleifen
  - die Abbruchbedingung steht im Schleifenfuß
  - wird auch als offene oder nicht abweisende Schleife bezeichnet
  - wird mind. 1 Mal durchlaufen
- zählende Schleifen
  - besitzt einen Zähler
  - hat i.d.R. eine feste Anzahl an Durchläufen
  - die Abbruchbedingung kann sich in Java nicht nur auf den Zähler beziehen

### Kopfgesteuerte Schleifen

```
while (ausdruck)
 anweisung;
```

- in Java realisiert durch die while-Schleife
- die Abbruchbedingung ausdruck muss als Ergebnis einen Wert vom Typ boolean zurückliefern
- ist das Ergebnis von ausdruck true, wird die Anweisung anweisung; ausgeführt
- liefert ausdruck als Ergebnis false, wird mit der ersten Anweisung nach der Schleife fortgefahren
- ist das Ergebnis von ausdruck gleich zu Beginn false, werden die Anweisungen in der Schleife nie ausgeführt ⇒ daher der Begriff: abweisende Schleife

### Fußgesteuerte Schleifen

```
do
 anweisung1;
while (ausdruck);
```

- in Java realisiert durch die do-while-Schleife
- zunächst wird anweisung1 ausgeführt
- erst danach wird das Ergebnis der Abbruchbedingung ausdruck geprüft
- ist das Ergebnis von ausdruck true, wird die Schleife ein weiteres Mal durchlaufen
- ist das Ergebnis von ausdruck false, wird die Schleifenverarbeitung beendet
- da die Bedingung erst nach dem ersten Durchlauf geprüft wird, muss die Schleife mindestens einmal durchlaufen werden ⇒ daher der Begriff: nicht abweisende Schleife

### Zählende Schleifen

```
for (init; test; update)
 anweisung1;
```

- in Java realisiert durch die for-Schleife
- der Schleifenkopf besteht aus drei optionalen Ausdrücken
  - der init-Teil wird i.d.R. zur Initialisierung von einem oder mehreren Schleifenzählern verwendet, die nur lokal innerhalb der Schleife bekannt sind
  - der test-Teil bildet die Abbruchbedingung der Schleife
  - anweisung1 wird nur dann ausgeführt, wenn test als Ergebnis true liefert
  - fehlt der test-Teil, setzt der Compiler an diese Stelle die Konstante true
  - im update-Teil werden die Schleifenzähler verändert
- besondere for-Schleife zum Durchlaufen von Feldern (Arrays)

```
for (Typ Bezeichner : Feld)
```

## Die break- und continue-Anweisung

- beide Anweisungen dienen der Änderung des Ablaufs innerhalb von Schleifen
- die break-Anweisung
  - eine break-Anweisung innerhalb einer Schleife führt zum Verlassen der Schleife
  - das Programm wird mit der ersten Anweisung nach der Schleife fortgesetzt
- die continue-Anweisung
  - der aktuelle Schleifendurchlauf wird beendet und das Programm springt zum Ende der Schleife
  - ein neuer Durchlauf der Schleife mit Prüfung der Abbruchbedingung beginnt

## E. Objektorientierung

### 1. Einführung in die Objektorientierung

#### Klassen

- Klassen bilden den Bauplan (die Schablone) für Objekte
- nach dem Bauplan können mehrere Objekte erzeugt werden

#### Objekte

- stellen die sog. Instanz einer Klasse dar
- können erzeugt, verändert und zerstört werden
- haben eine Identität und einen Zustand

#### Attribute

- beschreiben die Eigenschaften von Objekten
- beschreiben den Zustand eines Objektes

#### Methoden

- beschreiben das Verhalten von Objekten
- stellen die Funktionalität von Objekten dar

## Attribute und Methoden

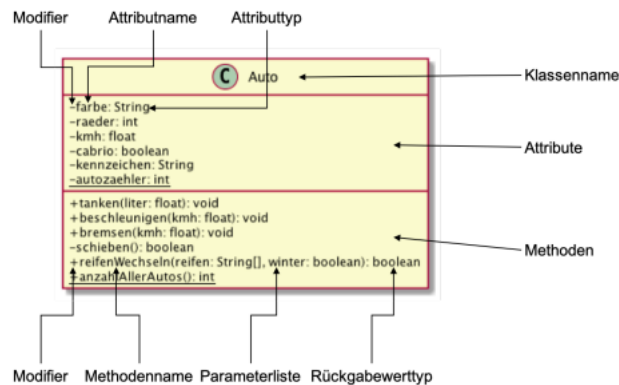
- Attribute sind Variable oder Konstanten innerhalb eines Objektes
- Attribute werden wie Variablen oder Konstanten deklariert
- in den Methoden werden die Algorithmen zu einem Objekt implementiert
- Methoden können einen Rückgabewert haben
- der Rückgabewert einer Methode kann von einem einfachen oder einem Referenzdatentyp oder vom Typ void sein
- Methoden vom Rückgabewerttyp void haben keinen Rückgabewert
- die Deklaration von Methoden erfolgt nach folgender Syntax

```
[Modifier] Typ Name ([Übergabeparameter]) {
 Anweisungen;
}
```

- sowohl Attribute und Methoden haben einen Modifier, der die Sichtbarkeit außerhalb der Klassen festlegt

## 2. Darstellung von Klassen in der UML

- UML steht für Unified Modelling Language
- wird im Rahmen der objektorientierten Analyse und im objektorientierten Design eingesetzt
- beschreibt eine standardisierte Sammlung von Diagrammen
- die Diagramme werden unterschieden in
  - statische Diagramme
  - dynamische Diagramme
- Klassen werden durch Klassendiagramme dargestellt
- Klassendiagramme zählen zu den statischen Diagrammen
- in den Klassendiagrammen werden festgelegt
  - der Name der Klasse
  - die Namen und Datentypen der Attribute
  - die Namen und Rückgabewerte der Methoden
  - die Sichtbarkeit von Attributen und Methoden



## 3. Objekte erzeugen

- Objekte sind Instanzen einer Klasse
- zum Erzeugen von Objekten muss zunächst eine Klasse definiert werden
  - die Definition einer Klasse beginnt mit dem Schlüsselwort class
  - innerhalb der Klasse werden die entsprechenden Attribute und Methoden zu der Klasse deklariert und implementiert
  - die Definition einer Klasse beschreibt also den Aufbau und das Verhalten einer Klasse
- Objekte werden wie folgt erzeugt
  - zunächst wird eine Variable vom Typ der Klasse deklariert
  - mit Hilfe des new-Operators wird ein neues Objekt der Klasse instanziiert und muss der Variable zugewiesen werden
  - durch Verwendung des new-Operators wird gleichzeitig für das neue Objekt Speicher allokiert
  - mit dem new-Operator wird eine spezielle Methode der Klasse implizit aufgerufen, der Konstruktor

#### 4. Konstruktoren

- sind spezielle Methoden zum Erzeugen von neuen Objekten einer Klasse
- Konstruktoren tragen den gleichen Namen wie die Klasse
- werden ausschließlich über den new-Operator aufgerufen
- liefern als Ergebnis die Referenz auf das neu geschaffene Objekt zurück
- es existiert ein Standardkonstruktor
  - dieser wird automatisch vom Compiler zur Verfügung gestellt, wenn in der Klassendefinition kein Konstruktor implementiert wurde
  - besitzt keine Übergabeparameter
  - sobald ein Konstruktor in der Klasse implementiert wurde, erzeugt der Compiler keinen Standardkonstruktor mehr
- Konstruktoren können dazu verwendet werden, um die Attribute zu initialisieren

#### 5. Prinzip der Kapselung

- die Kapselung ist ein Programmiergrundsatz in der objektorientierten Programmierung
- es handelt sich dabei um keine Eigenschaft der Programmiersprache Java
- Idee: auf die Attribute eines Objektes wird nicht direkt, sondern über spezielle Methoden zugegriffen
- dazu müssen die Attribute vor der Außenwelt verborgen und vor unerlaubtem Zugriff geschützt werden
- dies geschieht durch die Sichtbarkeitsmodifier
- Ziel: die Werte der Attribute dürfen nur im Sinne des Entwicklers sinnvoll verändert werden -> Plausibilitätsprüfung
- die speziellen Änderungsmethoden werden als Getter- und Setter-Methoden bezeichnet
  - Getter-Methoden dienen zum Auslesen der Attribute
  - Setter-Methoden werden zum Verändern der Attribute verwendet

#### 6. Sichtbarkeit von Attributen und Methoden

##### private

- nur innerhalb der Klasse sichtbar
- stärkste Form der Kapselung, da auf die Attribute und Methoden nur innerhalb der Klasse zugegriffen werden kann

##### protected

- nur innerhalb eines Paketes und in Subklassen (siehe Kapitel 6: Vererbung) sichtbar

##### default

- nur innerhalb des Paketes sichtbar

##### public

- von überall sichtbar
- normalerweise sind die Getter- und Setter-Methoden öffentlich, um auf private Attribute zuzugreifen

# Beispiel für Getter- und Setter-Methoden

```
package prog1.demos.objekt;
class Auto {

 // Deklaration der gekapselten Attribute
 private int ps;
 private float kmh;
 private String kfzKZ;
 private String marke;

 // Getter- und Setter-Methoden
 public String getKfzKZ() {
 return kfzKZ;
 }

 public void setKfzKZ(String kfzKZ) {
 this.kfzKZ = kfzKZ;
 }

 public float getKmh() {
 return kmh;
 }

 public void setKmh(float kmh) {
 this.kmh = kmh;
 }

}
```

## 7. Zugriff auf Attribute und Methoden

- auf Methoden und Attribute wird in der Punktnotation zugegriffen

```
objektname.methode(Übergabeparameter);
objektname.attribut;
```

- Voraussetzung: die Sichtbarkeit der Methoden und Attribute lässt den direkten Zugriff zu
- call by value
  - alle Parameter werden in Java mit call by value übergeben
  - Änderungen der Werte innerhalb der Methoden haben keinen Einfluss auf den Übergabeparameter
- call by reference
  - werden Referenzdatentypen (Objekte) übergeben, steht in der Methode die Referenz auf das Originalobjekt zur Verfügung
  - alle Methoden und Attribute zum Originalobjekt stehen in der Methode zur Verfügung
  - Änderungen der Attribute finden somit innerhalb des Originalobjektes statt

```
package prog1.demos.objekt;
class Auto {

 // Deklaration der Attribute
 private int ps;
 private float kmh;
 private String kfzKZ;
 private String marke;
 public int tueren;

 // Getter- und Setter-Methoden
 public String getKfzKZ() {
 return kfzKZ;
 }

 public void setKfzKZ(String kfzKZ) {
 this.kfzKZ = kfzKZ;
 }

 public float getKmh() {
 return kmh;
 }

 public void setKmh(float kmh) {
 this.kmh = kmh;
 }

}
```

```
package prog1.demos.objekt;
class AutoTest {
 public static void main(String[] args) {

 Auto bmw = new Auto();
 Auto audi = new Auto();

 bmw.tueren = 5;
 audi.tueren = 3;

 bmw.setKfzKZ("HD-XX 321");
 audi.setKmh(135.7f);

 System.out.println("Der BMW hat das
Kennzeichen: " + bmw.getKfzKZ());

 System.out.println("Der Audi fährt: " +
audi.getKmh());

 }
}
```

## 8. Überladene Methoden

- Methoden mit dem gleichen Namen werden innerhalb einer Klasse mehrmals definiert
- die Methoden unterscheiden sich dabei in der Anzahl und/oder in der Typisierung der Übergabeparameter
- die Modifier können dabei unterschiedliche Sichtbarkeiten zulassen
- Ziel: gleichnamige Operationen können mit unterschiedlichen Datentypen als Parameter versorgt werden
- gleichnamige Operationen können unterschiedliche Funktionalität zur Verfügung stellen
- auch Konstruktoren können überladen werden

```
package prog1.demos.objekt;
class Auto {

 // Deklaration der gekapselten Attribute
 private int ps;
 private float kmh;
 private String kfzKZ;
 private String marke;

 // überladene Konstruktoren und Methoden
 Auto(){
 ps = 75; kmh = 0;
 kfzKZ = "XX-XX 0000"; marke = "Eigenbau";
 }

 public Auto(int ps, float kmh, String kfzKZ, String marke){
 this.ps = ps; this.kmh = kmh;
 this.kfzKZ = kfzKZ; this.marke = marke;
 }

 void beschleunigen(float pluskmh){
 kmh += pluskmh;
 }
 protected void beschleunigen(){
 kmh+= 10;
 }
}
```

## 9. Variable Argumente – varargs

- können eine beliebige Anzahl von Werten als Argumente übergeben, einschließlich gar keiner.
- intern behandelt Java die varargs-Parameter als Array des angegebenen Typs. Daher können Sie die Methoden auf Arrays anwenden (wie Schleifen oder Längenüberprüfung).
- definition über `datentyp . . . name` in der Methodensignatur
- müssen am Ende der Übergabeparameter stehen
- nur ein variables Argument pro Methode

```
public class VarargsExample {
 public void sumOfNumbers(int... numbers) {
 int sum = 0;
 for (int number : numbers) {
 sum += number;
 }
 System.out.println("Summe: " + sum);
 }

 public static void main(String[] args) {
 VarargsExample sumUp = new VarargsExample();

 sumUp.sumOfNumbers(1); // Gibt: "Summe: 1"
 sumUp.sumOfNumbers(1, 2, 3); // Gibt: "Summe: 6"
 sumUp.sumOfNumbers(); // Gibt: "Summe: 0"
 }
}
```

## 10. Klassenattribute und -methode

- Klassenattribute und -methoden existieren unabhängig von einer Instanz einer Klasse
- ohne das ein Objekt existiert können diese verwendet werden
- sie werden durch den Modifier static als Klassenattribut bzw. -methode definiert
- ZU beachten: static-Methoden können nur auf static-Attribute zugreifen, und nicht auf Instanzattribute und -methoden
- auf Klassenattribute und -methoden wird ebenfalls in der Punktnotation über den Klassennamen zugegriffen
- Darstellung in UML: Klassenattribute und -methoden werden in UML durch Unterstreichug kenntlich gemacht

```
package prog1.demos.objekt;
class Auto {

 // Deklaration der gekapselten Attribute
 private int ps;
 private float kmh;
 private String kfzKZ;
 private String marke;
 private static int autoZaehler = 0;

 // Getter- und Setter-Methoden
 Auto(){
 autoZaehler++;
 ps = 75;
 kmh = 0;
 kfzKZ = "XX-XX 0000";
 marke = "Eigenbau";
 }

 public static int getAutoZaehler() {
 return autoZaehler;
 }

 public static void setAutoZaehler(int autoZaehler) {
 Auto.autoZaehler = autoZaehler;
 }
}
```

```
package prog1.demos.objekt;
class AutoTest {
 public static void main(String[] args) {

 System.out.println(Auto.getAutoZaehler());

 Auto bmw = new Auto();
 Auto audi = new Auto();

 System.out.println(Auto.getAutoZaehler());

 bmw.tueren = 5;
 audi.tueren = 3;

 bmw.setKfzKZ("HD-XX 321");
 audi.setKmh(135.7f);

 System.out.println("Der BMW hat das
Kennzeichen: " + bmw.getKfzKZ());

 System.out.println("Der Audi fährt: " +
 audi.getKmh());
 }
}
```

## 11. Objekte löschen mit dem Garbage-Collector

- nicht mehr benötigte Objekte belasten den Speicher und müssen daher „eingesammelt“ werden
- diese Aufgabe übernimmt der Garbage Collector
- benötigte Objekte sind dadurch gekennzeichnet, dass sie noch referenziert sind (reachable)
- sobald keine Referenzvariable mehr auf das Objekt verweist, wird das Objekt vom Garbage Collector zerstört (unreachable)
- Java verfügt über eine automatische Garbage Collection
- der Garbage Collector in Java startet, wenn die Virtual Machine für neue Objekte Speicherplatz benötigt
- explizit kann der Garbage Collector auch über den Befehl System.gc(); gestartet werden
- der Garbage Collector ruft den Destruktor eines Objektes
- (**Deprecated**) Destruktoren sind parameterlose Methoden mit dem Namen

```
protected void finalize();
```

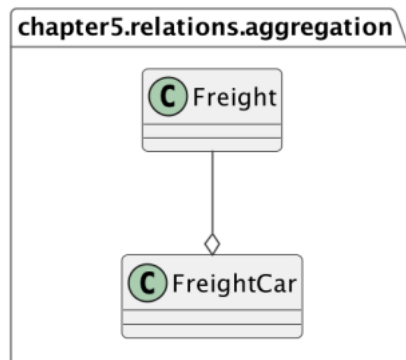
## 12. Beziehungen zwischen Objekten

### Assoziation

- beschreibt die Struktur einer Menge von Beziehungen zwischen Objekten
- binäre Assoziationen beschreiben die Beziehung zwischen je zwei Klassen
- Assoziationen können benannt werden, durch eine verbale Beschreibung der Beziehung
- die Darstellung in der UML erfolgt durch eine einfache Verbindungslinie zwischen den Klassen

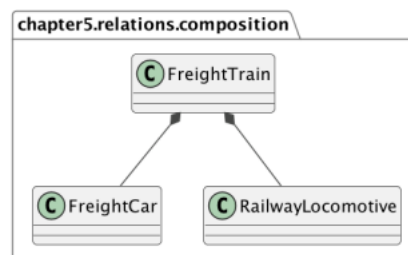
### Aggregation

- Aggregationen stellen eine spezielle Form der Assoziation dar
- Aggregationen stellen eine „Teile-Ganzes-Beziehung“ oder „Besteht-aus-Beziehung“ dar
- Aggregationen beschreiben keine existentielle Abhängigkeiten
- Beispiel: die Beziehung zwischen einem Güterwagen und der Fracht; der Wagon existiert auch ohne Fracht
- die Notation in der UML erfolgt durch eine Linie mit einer Raute am Ende, wobei die Raute bei der Aggregatsklasse (dem Ganzen) angesiedelt ist



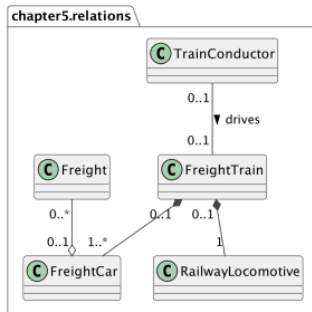
### Komposition

- die Komposition ist eine spezielle Form der Aggregation
- Kompositionen sind „strenger“ als Aggregationen
- es handelt sich ebenfalls um eine „Teile-Ganzes-Beziehung“
- die Existenz des Ganzen ist im Unterschied zur Aggregation von der Existenz des einzelnen Teils abhängig
- Beispiel: ein Güterzug kann nur dann existieren, wenn mind. eine Zuglokomotive und mind. ein Güterwagen existiert
- die Darstellung erfolgt analog der Aggregation nur mit einer ausgefüllten Raute



# Kardinalität (Multiplizität)

- Beschreibt die mengen- / zahlenmässige Ausprägung einer Beziehung (Assoziation)
- Werden an der Assoziation eingetragen



| Kardinalität | Beschreibung                                                                                                                      |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------|
| 1            | Ein Objekt muss genau einem anderen Objekt zugeordnet sein (MUSS-Beziehung, siehe Komposition)                                    |
| 0..1         | Es kann ein oder kein Objekt dem anderen zugeordnet sein (Kann-Beziehung)                                                         |
| 0..n         | Es können beliebig viele Objekte einem anderen Objekt zugeordnet sein (Kann-Beziehung)                                            |
| 1..n         | Es muss mindestens ein Objekt bis beliebig viele Objekte einem anderen Objekt zugeordnet sein (MUSS-Beziehung, siehe Komposition) |

## F. Objektorientierung

### Kontrollfragen I

1. Grenzen Sie die Begriffe Klasse und Objekt voneinander ab!
2. Was beschreiben Attribute und Methoden?
3. Wie werden Klassen in der UML dargestellt?
4. Wie werden in Java Objekte erzeugt? Welche Rolle spielen dabei die Konstruktoren einer Klasse?
5. Erläutern Sie den Begriff des Konstruktors!
6. Beschreiben Sie das Konzept der Kapselung in der objektorientierten Programmierung!
7. Welche unterschiedlichen Sichtbarkeitsmodifizier sind Ihnen bekannt? Beschreiben Sie die Sichtbarkeit jedes Modifiers!
8. Beschreiben Sie den Zugriff auf Instanzattribute und -methoden! Welche Rolle spielen dabei die Sichtbarkeitsmodifizier?
9. Unterscheiden Sie die Begriffe „call by value“ und „call by reference“!

### Kontrollfragen II

10. Was versteht man unter dem Begriff „überladen von Methoden“?
11. Worin unterscheiden sich Klassenattribute und -methoden von Instanzattributen und -methoden?
12. Erläutern Sie das Prinzip der Garbage Collection in Java!
13. Was ist der Destruktor und wie wird dieser aufgerufen?
14. Erläutern Sie die Begriffe Assoziation, Aggregation und Komposition! Wie werden die unterschiedlichen Beziehungen in der UML dargestellt?

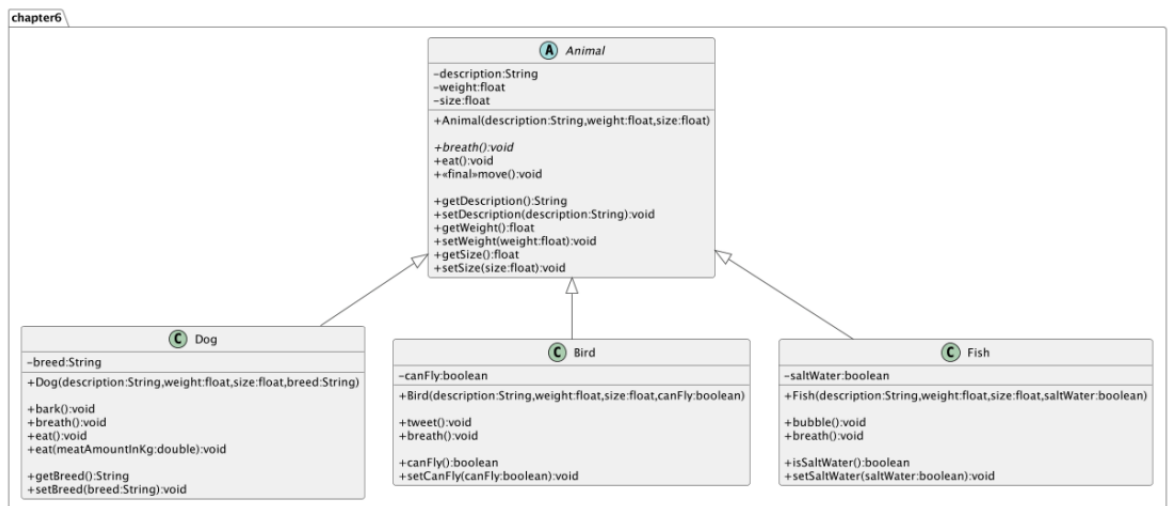
- **Klassenattribute und -methoden:** Gehören zur Klasse selbst, werden mit `static` deklariert, und sind für alle Instanzen der Klasse gemeinsam.
- **Instanzattribute und -methoden:** Gehören zu einer bestimmten Instanz, werden ohne `static` deklariert, und jede Instanz hat ihre eigenen Kopien.

## G. Vererbung

### 1. Eigenschaften der Vererbung

- eine Unterklasse (Subklasse) erbt die Attribute und Methoden einer Oberklasse (Superklasse)
- statt Vererbung spricht man auch vom Ableiten von Klassen? eine Subklasse leitet sich aus einer oder mehreren Superklassen ab
- es entsteht eine Vererbungshierarchie, die theoretisch beliebig tief geschachtelt werden kann
- i.d.R. besitzt die Subklasse noch zusätzliche Attribute und Methoden
- Ziel und Vorteil: bestehender Programmcode kann wieder verwendet werden (Reuse)
- Mehrfachvererbung besagt, dass eine Subklasse von mehreren Superklassen erbt
- Superklassen stellen eine Generalisierung ihrer Subklassen dar
- umgekehrt sind Subklassen Spezialisierungen ihrer Superklassen

## Darstellung der Vererbung in UML



### 2. Vererbung in Java

- Java unterstützt nur die Einfach- und nicht die Mehrfachvererbung
- über Interfaces (Kapitel 7) kann auf mehrere Klassen Bezug genommen werden
- die Vererbung ist durch folgende Syntax definiert:

```
class Subklasse extends Superklasse { }
```

- mit `extends` verweist die Subklasse auf ihre Superklasse
- nach `extends` darf lediglich eine Superklasse angegeben werden
- alle Attribute und Methoden mit Ausnahme der Konstruktoren der Superklasse werden an die Subklasse vererbt
- in der Subklasse kann die Funktionalität der abgeleiteten Klasse erweitert und verändert werden
  - Hinzufügen von Methoden und Attributen
  - Überladen von Methoden
  - Überschreiben von Methoden

### 3. Superklasse Object

- ist die Wurzel der Klassenhierarchie in Java
- liegt im Paket java.lang
- von ihr sind alle Klassen explizit oder implizit abgeleitet, d.h. jede eigene Klasse ist immer eine Subklasse von Object
- damit stehen alle Methoden der Klasse Object in allen abgeleiteten Klassen zur Verfügung
  - Object() – parameterloser Konstruktor
  - toString() – Konvertierung eines Objekts in einen String ausgeführt
  - equals() – vergleicht zwei Objekte miteinander
  - hashCode() – berechnet den Hashwert eines Objektes
  - clone() – kopiert zwei Objekte
  - finalize() – ist der Destruktor
- in Referenzvariablen vom Typ Object können alle beliebigen Objektreferenzen gespeichert werden (vgl. Narrowing cast)

### 4. Methoden überschreiben

- Methoden, die bereits in der Superklasse implementiert sind, können in Subklassen neu definiert werden (Überschreiben)
- zur Laufzeit wird entschieden, welche Methode konkret ausgeführt wird (dynamisches Binden)
- dabei sucht die JVM zunächst in der Klasse des Referenzdatentyps nach einer passenden Methode
- wird dort keine passende Methode gefunden, wird die Vererbungshierarchie von unten nach oben nach einer passenden Methode durchsucht
- dies erfordert zusätzliche Rechnerkapazität und wirkt sich daher negativ auf die Performance aus
- das Überschreiben kann durch zwei Modifier vermieden werden
  - private ⇒ Methode ist in Subklasse nicht sichtbar
  - final ⇒ verhindert explizit das Überschreiben
- die Sichtbarkeit bei überschriebenen Methoden darf erhöht aber nicht eingeschränkt werden

### 5. Weitere Modifier

#### abstract

- abstrakte Klassen sind i.d.R. noch nicht ausreichend spezialisiert und dienen nur als grobe Vorlage für Subklassen
- von abstrakten Klassen können keine Objekte erzeugt werden
- beinhaltet eine Klasse mind. eine abstrakte Methode, so muss die Klasse abstrakt werden
- abstrakte Methoden beinhalten nur den Methodenkopf, aber keine Implementierung im Methodenrumpf
- Subklassen von abstrakten Superklassen müssen alle abstrakten Methoden der Superklasse implementieren (konkrete Klasse) oder selbst abstrakt werden
- UML: Abstrakte-Klassen und -Methoden werden in UML durch kursiv-Schreibweise kenntlich gemacht

#### final

- Klassen, die mit dem Modifier final versehen sind, dürfen nicht weiter abgeleitet werden
- Methoden mit dem Modifier final dürfen nicht überschrieben werden final-Attribute sind Konstanten, deren Wert sich nicht verändern darf

### 6. Bedeutung von this und super

- mit dem Ausdruck super kann aus einer abgeleiteten Klasse mithilfe der Punktnotation auf Attribute und Methoden der Superklasse zugegriffen werden
- ein kaskadierender Aufruf super.super.x() ist nicht möglich
- this ist eine Referenzvariable, die beim Anlegen des Objekts automatisch erzeugt wird und auf das aktuelle Objekt zeigt
- über die this-Referenz können eigene Methoden, Instanz- und Klassenattribute angesprochen werden
- mithilfe von this(Übergabeparamter) oder super(Übergabeparameter) können Konstruktoren verkettet werden
- this() und super() müssen bei verketteten Konstruktoraufrufen immer die als erste Anweisung im Konstruktor stehen
- this() verweist auf Konstruktoren der aktuellen und super() auf Konstruktoren der Superklasse

## 7. Casting von Referenztypen

### Narrowing Cast

- Objekte von Subklassen können in Referenzvariablen gespeichert werden, die vom Typ ihrer Superklasse sind
- dabei wird nur die Sicht auf die Objekte beschränkt, d.h. es sind nur noch die Attribute und Methoden sichtbar, die in der Superklasse deklariert sind
- die subklassen-spezifischen Attribute und Methoden sind noch vorhanden, aber vorübergehend ausgeblendet
- der narrowing cast beschreibt also den Wechsel von einer Sicht mit mehr auf eine Sicht mit weniger Details bei einem Objekt
- der narrowing cast ist ein wesentliches Prinzip im Vererbungskonzept von Java
- der narrowing cast ist eine wesentliche Voraussetzung für die Polymorphie

Ist ein Widening Cast eine sichere Cast??

### Widening Cast

- stellt die Umkehrung des narrowing cast dar
- Referenzen auf Objekte, die in einer Referenzvariable vom Typ der Superklasse gespeichert sind, sollen in einer Referenzvariable vom Typ der Subklasse gespeichert werden
- es handelt sich dabei um eine unsichere Konvertierung
  - es muss sichergestellt werden, dass es sich bei den Referenzen um Referenzen auf Objekte der Subklasse handelt
  - vor der Umwandlung muss der Typ der Referenz mit dem Operator `instanceof` überprüft werden  $\Rightarrow$  `referenzvariable instanceof Subklasse` muss `true` ergeben
  - bei der Durchführung muss ein expliziter Cast auf den Typ der Subklasse durchgeführt werden

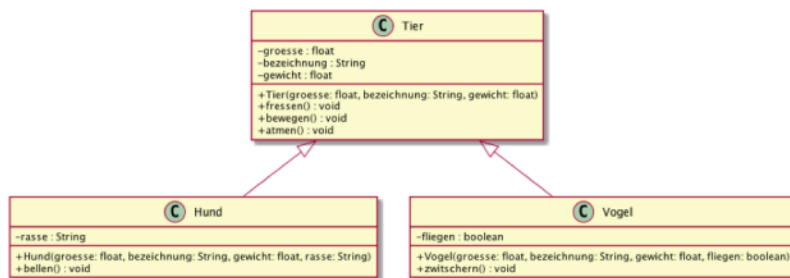
```
refSubklasse = (Subklasse)referenzdatentyp;
```

```
* der widening cast wechselt von einer Sicht mit weniger auf eine Sicht mit mehr Details und blendet die
</div><!-- element style="font-size: 0.8em;" -->
```

## 8. Polymorphie

# Übersicht

- neben der Kapselung und der Vererbung ein weiteres Grundkonzept in der objektorientierten Programmierung
- bedeutet die Vielgestaltigkeit von Objekten
- basiert auf dem Konzept des dynamischen Bindens
- gleiche Methodenaufrufe rufen unterschiedliche Verhaltensweisen der Objekte hervor
- Beispiel:



## Beispiel Quellcode

```
package prog1.demos.objekt;

class AutoTest {
 public static void main(String[] args) {

 Tier[] x = new Tier[2];

 x[0] = new Hund(25.5f, "Bello", 15.8f, "Schäferhund"); //Narrowing Cast
 x[1] = new Vogel(10.4f, "Tweety", 0.4f, true); //Narrowing Cast

 x[0].atmen();
 x[1].atmen();
 }
}
```

## Kontrollfragen I

1. Beschreiben Sie das objektorientierte Konzept der Vererbung!
2. Erläutern Sie die Begriffe Superklasse, Subklasse und Vererbungshierarchie!
3. Beschreiben Sie die Darstellung der Vererbung in der UML mithilfe eines kleinen Beispiels!
4. Was ist die Besonderheit in der Vererbung bei Java und wie ist die Vererbung syntaktisch in Java realisiert?
5. Beschreiben Sie die wesentlichen Eigenschaften der Klasse Object und ihre besondere Rolle in der Vererbungshierarchie in Java?
6. Beschreiben Sie das Konzept des Überschreibens von Methoden!
7. Was bewirken die Modifier abstract und final in Bezug auf Klassen?
8. Welche Auswirkungen haben die Modifier abstract und final bei Methoden?

## Kontrollfragen II

9. Wozu wird der Ausdruck super in der Vererbung benötigt?
10. Was stellt der Ausdruck this dar?
11. Beschreiben Sie das Konzept des narrowing und widening Cast!
12. Beschreiben Sie das Konzept des Polymorphismus in der objektorientierten Programmierung anhand eines einfachen Beispiels!

## H. Interfaces

### 1. Eigenschaften von Interfaces

## Eigenschaften von Interfaces

- Interfaces stellen eine besondere Form der Klassen dar
- sie definieren lediglich den Aufbau der Schnittstelle
  - Name von Methoden
  - Signaturen der Methoden
  - Attribute
- Interfaces beinhalten keine eigene Funktionalität (Standard-Implementierungen seit Java 8)
- die Funktionalität wird in Klassen implementiert, die das Interface nutzen wollen
- sie beinhalten ausschließlich abstrakte öffentliche Methoden
- alle Attribute sind als Konstanten festgelegt
- Deklaration von Interfaces mit dem Schlüsselwort interface
- Interfaces können von anderen Interfaces mit extends abgeleitet (vererbt) werden
- Interfaces können genau wie Klassen als Datentyp für Referenzvariablen genutzt werden

### 2. Implementierung von Interfaces

- um zu einem Interface die Funktionalität zur Verfügung zu stellen, muss eine Klasse das Interface implementieren
- dies erfolgt durch den Zusatz implements zur class-Anweisung z.B.

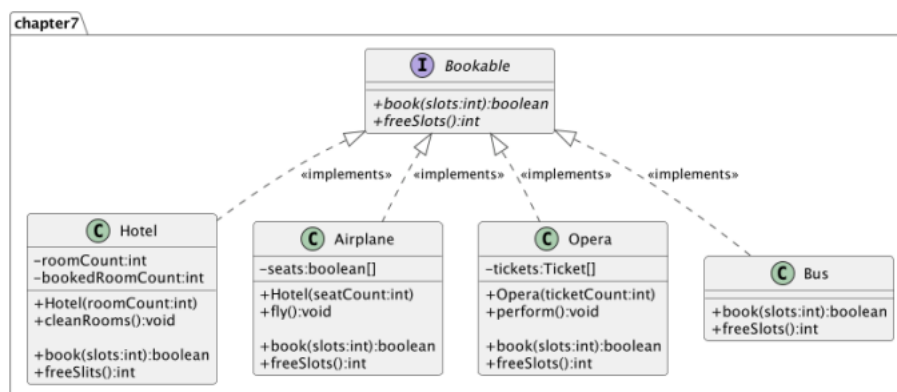
```
class xyz implements abc { }
```

- die Klasse muss alle Methoden des Interfaces implementieren oder als abstrakte Klasse deklariert werden
- Klassen können im Gegensatz zur Vererbung mehrere Interfaces implementieren z.B.

```
class xyz implements abc, def { }
```

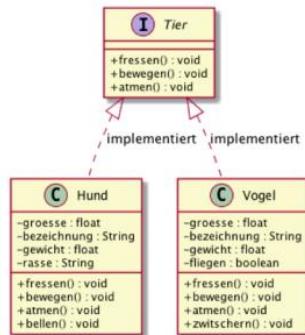
- durch die Implementierung mehrerer Interfaces wird eine Art von Mehrfachvererbung realisiert
- Objekte einer Klasse sind zu allen Interface-Datentypen kompatibel, sobald die Klasse das Interface implementiert z.B. alle Objekte der Klasse xyz könnten in Referenzvariablen vom Typ abc oder def gespeichert werden

## Darstellung in UML



### 3. Polymorphie über Interfaces

- analog der Polymorphie durch Vererbung
- Unterschied
  - Polymorphie durch Vererbung: Objekte einer Subklasse werden in Referenzvariablen vom Typ der Superklasse gespeichert
  - Polymorphie durch Interfaces: Objekte der implementierenden Klasse werden in Referenzvariablen vom Typ des implementierten Interfaces gespeichert
- Beispiel:



### FRAGE

#### 1. Unterschiede zwischen Interface und abstract class

##### + Abstract class

- ➔ **Vererbung:** eine Klasse kann nur eine einzige abstrakte Klasse erweitern. (Einzelvererbung)
- ➔ **Methoden:** abstrakte Klassen können sowohl abstrakte Methode als auch konkrete Methode enthalten.
- ➔ **Attribute:** abstrakte Klassen können Attribute haben und diese können auch konkrete Implementierung enthalten.
- ➔ **Konstruktoren:** Abstrakte Klassen können Konstruktoren haben, die von Unterklassen aufgerufen werden können.
- ➔ **Zweck:** Abstrakte werden verwendet, um eine gemeinsame Basis für verwandte Klassen zu schaffen und können teilweise implementierte Methoden enthalten

##### + Interface:

- ➔ **Vererbung:** eine Klasse kann mehrere Interfaces implementieren. (Mehrfachvererbung)
- ➔ **Methoden:** Interfaces enthalten standardmäßig nur abstrakte Methoden (ohne Implementierung). Seit Java 8 können Interfaces auch Standardmethoden (mit Implementierung) und statische Methoden enthalten.
- ➔ **Attribute:** Interfaces können nur Konstanten enthalten. (static final)
- ➔ **Konstruktoren:** Interfaces können keine Konstruktoren haben.
- ➔ **Zweck:** Interfaces werden verwendet, um eine Reihe von Methoden zu definieren, die von einer Klasse implementiert werden müssen, unabhängig von der Klassenhierarchie.